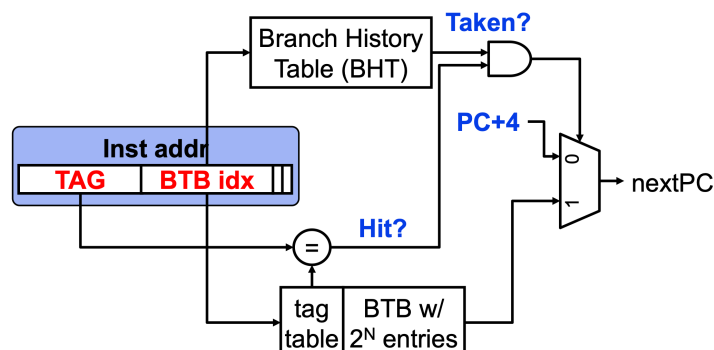


lab6_report

: 2022056562 서민용, 2023003227 이한주

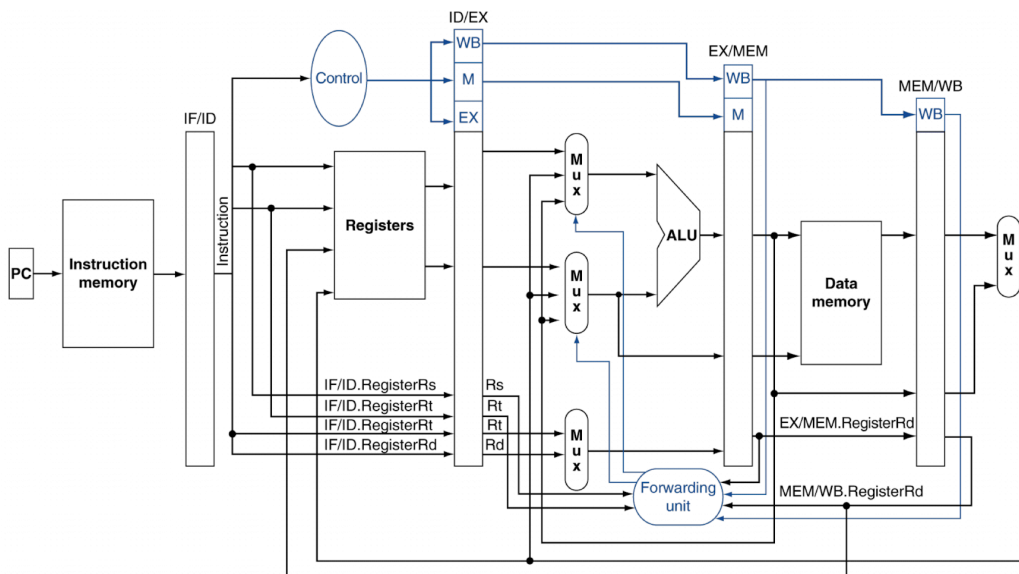
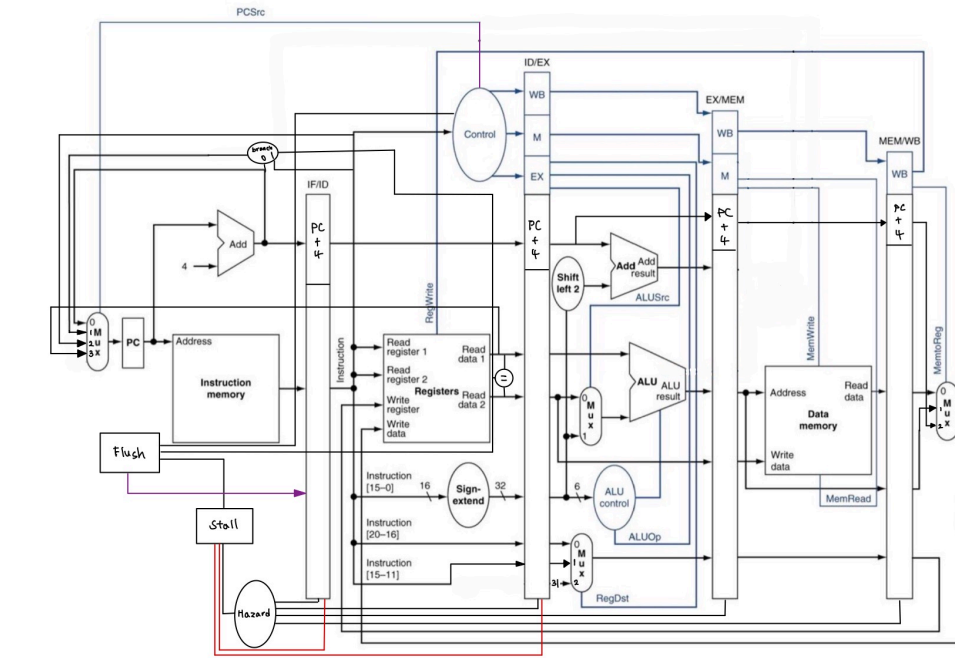
1. Overall structure

1-1. Branch Predictor [BTB + BHT]



- 이제 단순히 always not taken이 아닌, BTB와 BHT를 활용해서 더 잘 예측을 해야합니다.
- 과제에 명시해주신대로,
 - BTB는 LSB 2bit를 제외하고 6bit 사용하여 총 64개의 entry를 가지도록 했습니다.
 - BHT 역시 LSB 2bit를 제외하고 8bit를 사용하여 총 256개의 entry를 가지도록 했습니다.
 - 또한 공부할 땐 크게 신경을 안 썼던 부분인데, 같은 branch inst가 어찌다가 같은 entry로 indexing 된다면, 기존 entry를 evict시킵니다.

1-2. CPU Microarchitecture



- 기존 lab5의 data path 이외에도, 이제 Forwarding unit이 필요합니다. [그림은 시간이 없어서 추가로 그리진 못했습니다.]
 - 전체적인 구조는 lab5의 pipelined CPU와 완전히 같은데,
 - Forwarding 로직이 추가되어 stall을 덜 해도 되고,
 - Branch prediction이 더 smart해져서 flush를 조금 더 덜 하게 된다는 차이점이 있습니다.
- Design에 있어 하나 고려했던 점은, Forwarding unit을 따로 module 로 빼지 않고, microarchitecture와 같이 CPU.v 에서 자체적으로 Forwarding을 구현해주었습니다.
- 그 이유는 기존 lab5를 구현할 때 Hazard.v 를 따로 짜게 되니 해당 .v 를 구현하는건 수월했지만, 분리 되어 있으니 CPU.v 에서 연결해줄 때 오히려 헷갈렸던 경험 때문입니다.

- 사실 크게 차이가 없을 듯 하지만, 그냥 한번에 짜면 두 번 나누어서 고민하지 않아도 된다는 programmer로서의 입장에서 이와 같이 구현했습니다.

2. Implementation

- 전체적인 구조는 lab5와 완전히 동일하기 때문에, 큰 변경점이 있는 부분만 작성해보겠습니다.

2-1. BP.v

```
module BP (
    input clk,
    input rst,
    input [31:0] pc,
    output pred_taken,
    output [31:0] pred_target,
    input update_en,
    input [31:0] update_pc,
    input update_type,
    input update_taken,
    input [31:0] update_target
);
```

```
    assign idx = pc[7:2];
    assign tag = pc[31:8];
    assign pht_idx = pc[9:2];
    assign upd_idx = update_pc[7:2];
    assign upd_tag = update_pc[31:8];
    assign upd_pht_idx = update_pc[9:2];
    assign btb_hit = btb_valid[idx] && (btb_tag[idx] == tag);
    assign pht_taken = pht[pht_idx][1];
    assign pred_taken = btb_hit && (btb_type[idx] ? 1 : pht_taken);
    assign pred_target = btb_hit ? btb_target[idx] : 32'b0;
```

- Branch Predictor module에서는 CPU에서 현재 PC값과, 현재 CPU 내부에서 control inst 가 발생했고, 실제로 taken 되었는지, 아닌지에 대한 정보를 input으로 받고, output으로는 실제 control inst에 대해서 T/NT, T라면 target 값을 내보냅니다.
- 또한 BTB와 PHT가 함께 있어서, 들어온 input에 대해서 필요한 정보로 분해하여 사용합니다.
- 또한 BTB는 2bit를 추가적으로 보유하고 있어야하는데,

```
    reg btb_valid [0:63];
    reg btb_type [0:63];
    // ...
    assign btb_hit = btb_valid[idx] && (btb_tag[idx] == tag);
    assign pht_taken = pht[pht_idx][1];
    assign pred_taken = btb_hit && (btb_type[idx] ? 1 : pht_taken);
    assign pred_target = btb_hit ? btb_target[idx] : 32'b0;
```

- 2D array로 구현하니 오히려 헷갈려서 valid bit과 type bit으로 분리해서 사용했습니다.
- 추가적으로, PHT_Taken 같은 경우에 Jump inst인 경우는 그냥 1로 처리했습니다.

- 구현은 간단해서, BP 내부적으로는 enable이 들어올 때 update 해주는 것만 작성해주었습니다.

```
always @(posedge clk) begin
    // ...
    if (update_en) begin
        btb_valid[upd_idx] <= 1'b1;
        btb_type[upd_idx] <= update_type;
        btb_tag[upd_idx] <= upd_tag;
        btb_target[upd_idx] <= update_target;
        if (update_taken) if (pht[upd_pht_idx] != 2'b11) pht[upd_pht_idx] <= pht[upd_pht_idx] + 2'b01;
        else if (pht[upd_pht_idx] != 2'b00) pht[upd_pht_idx] <= pht[upd_pht_idx] - 2'b01;
    end
end
endmodule
```

2-2. HAZARD.v

- HAZARD unit의 핵심적인 차이점은, Forwarding이 보장되기 때문에, stall 하는 경우가 매우 줄었다는 점입니다.
 - 즉, dist == 3 인 hazard는 이제 사라졌기에 일부 input은 이제 필요없습니다.
- 그러나 저희는 lab5에서부터 branch early resolution을 적용시켰기 때문에, 수업에서 다루었던 것처럼 side effect에 대한 점을 고려해주어야합니다.

Side effect: Data hazard analysis

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID				use		use
EX	use produce	use	use	use		
MEM		produce				
WB						

- The branch instruction no longer compute (use) the register values at EX stage (but earlier at ID stage)
 - Consider (1) R/I-type instruction before branch and (2) LW instruction before branch
- 한마디로, lw 직후의 branch inst는 2 cycle, lw 이후 다른 inst 이후의 branch inst는 1 cycle, lw 이외의 inst 직후의 branch inst는 1 cycle stall 해야합니다.
 - 하여 lw를 detect 하기 위해서 lab5에 없던 input이 몇 가지 필요합니다. [MemtoReg_EX, MemtoReg_MEM. c.f. MemtoReg가 00 이라면 lw]

- 또한 branch인가를 확인해야하기 때문에, opcode의 case에서 추가적인 로직이 필요합니다.
- 이외의 코드는 이전 보다 더 간단해집니다.

```
always @(*) begin
    use_reg    = 2'b00;
    is_branch  = 1'b0;
    if (valid) begin
        case (opcode)
            // ...
            `OP_BEQ, `OP_BN
E: begin
                use_reg =
2'b11;
                is_branch =
1'b1;
            end
        endcase
    end
end
```

```
always @(*) begin
    stall = 2'b00;
    if (valid) begin
        if (is_branch) begin
n
            if (dep_rs_EX |
| dep_rt_EX) begin
                if (lw_in_E
X) stall = `STALL2;
            else stall
= `STALL1;
        end
        else if ((dep_r
s_MEM || dep_rt_MEM) && lw_in_M
EM) stall = `STALL1;
        end
        else begin
            if ((dep_rs_EX
|| dep_rt_EX) && lw_in_EX) stal
l = `STALL1;
        end
    end
end
```

2-3. RF.v

- 이제 Internal Forwarding이 지원되기 때문에, 간단한 수정사항이 있습니다.
- PPT에 나와있는 Pseudo Code 대로만 구현해주었습니다.

```
always @(*) begin
    if (RegWrite && (wr_addr == rd_
addr1)) rd_data1 = wr_data;
    else rd_data1 = register_file[r
d_addr1];

    if (RegWrite && (wr_addr == rd_
addr2)) rd_data2 = wr_data;
    else rd_data2 = register_file[r
d_addr2];
end
```

Internal RF forwarding

♦ We may design an RF that internally forwards data

```
// Verilog-style pseudocode
module RF (clk, raddr, rdata, waddr, wdata);
    // port declaration
    ...
    // storage
    reg [31:0] RF_data [0:31];

    always @(*) begin
        if (waddr == raddr1) rdata = wdata;
        else rdata = RF_data[raddr];
    end

    always @(posedge clk) begin
        // write RF
        RF_data[waddr] <= wdata;
    end
endmodule
```

2-4. CPU.v

- 이제 CPU 내부적으로 branch early resolution과 이를 통한 결과 값을 BP에 전달해주어야하고,
- Forwarding을 구현해주어야 합니다.
- 또한, branch 역시 rd_data1, rd_data2를 모두 사용하는 instruction 이기 때문에, branch로의 forwarding 역시 직접 구현해야합니다.
- 다른 module에서 가정한 상황을 직접 만들어주어야 합니다. 그렇기에 가장 변경점이 많은 파일입니다.

2-4-1. BP Logic

```
assign actual_target =
(id_PCSrc == 2'd3) ? rd_data1 :
(id_PCSrc == 2'd2) ? id_jump_target :
branch_taken ? id_branch_target : IFID_PC;

assign mispredict =
id_valid_inst && !id_halt && !stall_active &&
(id_PCSrc != 2'd0) &&
((IFID_PredTaken != control_taken) ||
(IFID_PredTaken && control_taken && (IFID_PredTarget != actual_target)));

assign jr_inst = id_valid_inst && !id_halt && (id_PCSrc == 2'd3);

assign control_flush = !stall_active && (mispredict || jr_inst);

assign update_enable = id_valid_inst && !id_halt && !stall_active &&
(id_PCSrc == 2'd1 || id_PCSrc == 2'd2);

assign update_target = (id_PCSrc == 2'd2) ? id_jump_target :
branch_taken ? id_branch_target : IFID_PC;

// ...
always @(posedge clk) begin
    // ...
    if (control_flush) begin
        IFID_Valid      <= 1'b0;
        IFID_PC         <= 32'd0;
        IFID_Inst        <= 32'd0;
        IFID_PredTaken   <= 1'b0;
        IFID_PredTarget  <= 32'd0;
    end
    else begin
        IFID_Valid      <= 1'b1;
    end
end
```

```

        IFID_PC          <= PC + 4;
        IFID_Inst        <= inst;
        IFID_PredTaken   <= bp_pred_taken;
        IFID_PredTarget  <= bp_pred_target;
    end

```

- `actual_target` : 현 inst 기준, 실제 다음 target PC를 계산합니다.
- `mispredict` : $N \neq NT$, $NT \neq T$, 그리고 $T == T$ 인데, BP가 예측한 target이 실제 target과 다른 경우를 고려합니다.
- `jr_inst` : BP가 JR 같은 경우는 단순 flush를 시키는 로직으로 짜여져있기 때문에, 이를 구분해줍니다.
- `control_flush` : 실제로 flush를 해야하는가를 계산합니다.
- `update_enable, update_target` : 예측한 값에 대해서 BP를 update 해주기 위한 wire 들입니다.
- `IFID_PredTaken/Target, bp_pred_taken/target` :
 - 전자는 branch resolution이 일어나는 단계에 임시 저장해줄, 새롭게 추가된 latch이고,
 - 후자는 BP module의 output과 연결된 wire 값들 입니다.
 - bp_pred_taken/target 같은 경우, wire 문법 상 default 값이 필요한데, **사실 쓸모없는 값이긴 합니다. [enable signal에 의해서 쓰이지 않게되므로]**
 - 그렇다고 32'b0 같이 명시적으로 넣어주자니 정말 어찌다가 32'b0이 BTB에 저장되고 그걸 예측 값으로 사용하게 된다면,
 - program이 채 끝나지도 않았는데 먼저 halt 가 될 수 있지 않나 싶어
 - 만일 오작동으로 쓰이게 되더라도, 프로그램 자체에는 영향 없고 기능 저하만 생기도록 현재 PC 값을 default 값으로 사용해주었습니다.

2-4-2. Forwarding Logic & branch early resolution Logic

```

assign MEMWB_Result =
    (MEMWB_MemtoReg == 2'd0) ? MEMWB_Data :
    (MEMWB_MemtoReg == 2'd1) ? MEMWB_ALUResult : MEMWB_PC;

assign operand1_fwd =
    (ForwardA == 2'b01) ? EXMEM_ALUResult :
    (ForwardA == 2'b10) ? MEMWB_Result : IDEX_Data1;

assign operand2_fwd =
    (ForwardB == 2'b01) ? EXMEM_ALUResult :
    (ForwardB == 2'b10) ? MEMWB_Result : IDEX_Data2;

always @(*) begin
    if (EXMEM_RegWrite && (EXMEM_RegAddr == IDEX_RS)) ForwardA =
    2'b01;

```

```

        else if (MEMWB_RegWrite && (MEMWB_RegAddr == IDEX_RS)) Forward
A = 2'b10;
        else ForwardA = 2'b00;

        if (EXMEM_RegWrite && (EXMEM_RegAddr == IDEX_RT)) ForwardB =
2'b01;
        else if (MEMWB_RegWrite && (MEMWB_RegAddr == IDEX_RT)) Forward
B = 2'b10;
        else ForwardB = 2'b00;

        if (EXMEM_RegWrite && (EXMEM_RegAddr == rs)) branch_data1 = EX
MEM_ALUResult;
        else if (MEMWB_RegWrite && (MEMWB_RegAddr == rs)) branch_data1
= MEMWB_Result;
        else branch_data1 = rd_data1;

        if (EXMEM_RegWrite && (EXMEM_RegAddr == rt)) branch_data2 = EX
MEM_ALUResult;
        else if (MEMWB_RegWrite && (MEMWB_RegAddr == rt)) branch_data2
= MEMWB_Result;
        else branch_data2 = rd_data2;

        // ...
        operand2 = IDEX_ALUSrc ? ext_imm : operand2_fwd;

```

- MEMWB_Result : mux를 통해서 RF로 어떤 값이 쓰여지는지 구해둡니다. lw 라면 Data를 RF에 쓰는 것이고, 그 이외의 ALU operation이면 ALUResult 를 RF에 쓰는 것이고, JR이라면 PC 값을 RF에 쓰는 것입니다.
- operand1/2_fwd : ALU operation 시, operand 인 rs에 해당되는 부분이 forwarding 신호에 따라서 어디서 forwarding 을 받을지 결정합니다.

Forwarding Logic

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

- 로직은 수업 슬라이드와 동일하게 작성했습니다.
- combinational circuit에서 Forwarding signal을 결정해줍니다.
- 이 역시 수업 슬라이드와 동일하게 작성했습니다.

Forwarding Logic

```

if (rs_EX!=0) && (rs_EX==dest_MEM) && RegWrite_MEM then
    forward operand from MEM stage // dist=1
else if (rs_EX!=0) && (rs_EX==dest_WB) && RegWrite_WB then
    forward operand from WB stage // dist=2
else
    use the operand from register file // dist >= 3

```

- 허나 수업에선 rs_EX라는 help function을 통해서 실제로 EX stage에서 rs를 사용하는가? 까지 체크해서 redundant 한 Forwarding이 없도록 해주었는데,
- 저희 구현에서는 그냥 제외해주었습니다.
- 어쨌피 operand를 사용하지 않을 것이기 때문에, Forwarding이 되던 말던 오류가 없기 때문입니다.
- 사실 현실적으로는 전혀 합리적이지 못한 선택지이고, verilog level 이라서 가능한 구현이라고 생각합니다...
- 추가적으로, branch는 forwarding logic과 별개로, Ex stage에서 연산을 하지 않기 때문에, branch 인 경우를 대비하여 branch_data1/2 라는 변수로 forwarding을 직접 해주고, branch resolution에서 rd_data1/2 가 아닌 이 값을 사용하도록 수정해주었습니다.

```

assign branch_taken =
    id_valid_inst && !id_halt &&
    (((opcode == `OP_BEQ) && (branch_data1 == branch_data2)) ||
    ((opcode == `OP_BNE) && (branch_data1 != branch_data2)));

```

3. Result

<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 6265000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>	<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 2205000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>
<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 1725000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>	<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 8485000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>
<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 3235000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>	<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 2785000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>
<pre> minyongseo@MinYong-MacBook-Air HW6_test % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 5125000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW6_test % </pre>	

- 주어진 테스트케이스에 대해서 모두 잘 통과한 모습입니다.
 - 물론 이번 과제는 구조적으로 완전히 치밀하게 작성하지 못해서, 또 다른 edge case가 있을 때 오류가 있을 수 있겠다는 생각도 듭니다.
- 기존 lab5의 속도와 비교해보았을 때, 7번 테스트케이스를 제외하고는 모두 control inst가 있기 때문에, 높은 성능 향상 폭을 보입니다.

<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 11185000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>	<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 4125000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>
<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 2535000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>	<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 17775000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>
<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 5545000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>	<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 4995000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>
<pre> minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out Program Terminate Simulation success!!! CPU_tb.v:57: \$finish called at 12785000 (1ps) ❖ minyongseo@MinYong-MacBook-Air HW05_HanJoo % </pre>	

lab5 Result

- 또한, 7번 테스트케이스 같은 경우엔 매우 dependency가 강한 ALU inst 들로 구성 되어있어 가장 큰 성능 향상인 2배 이상을 보이는 듯 합니다.

- 제 예상으로는 mis prediction으로 인한 성능 저하가 개선됨으로써 7번 테스트케이스를 제외한 나머지의 성능 향상이 훨씬 높을 것이라고 생각했는데, Forwarding의 영향도 만만치 않았던 것 같습니다.